



VPC Peering



jjoseph1608@outlook.com

Accept VPC peering connection request

Info

Are you sure you want to accept this VPC peering connection request? (pcx-0fc0d44c414dd9ef8 / VPC 1 <-> VPC 2)

Requester VPC

vpc-08f05b33c825a4e4c / NextWork-1-vpc

Accepter CIDRs

-

Requester owner ID

 304152263329(This account)

Accepter VPC

vpc-0c8f254bc8f982e9d / NextWork-2-vpc

Requester Region

Stockholm (eu-north-1)

Accepter owner ID

 304152263329(This account)

Requester CIDRs

 10.1.0.0/16

Accepter Region

Stockholm (eu-north-1)

Cancel

Accept request



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that lets you create isolated private networks within AWS, and it is useful because it gives you full control over your network environment including IP address ranges, subnets, route tables, and security settings, keeping your cloud resources separate and secure.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create two isolated networks and connect them together using a VPC peering connection, demonstrating how separate cloud environments can communicate with each other over private IP addresses

One thing I didn't expect in this project was...

One thing I didn't expect in this project was having to troubleshoot multiple errors just to get a simple ping working, from missing public IPs to security group rules blocking ICMP traffic. It was a good reminder that networking has a lot of moving parts that all need to be configured correctly

This project took me...

This project took me roughly 30 minutes



In the first part of my project...

Step 1 - Set up my VPC

In this step, I will create two VPCs from scratch using the visual VPC resource map, because these will be the two isolated networks I'll later connect together with a peering connection

Step 2 - Create a Peering Connection

In this step, I will set up a VPC peering connection between my two VPCs, because this is the link that will allow the two isolated networks to route traffic to each other using private IPs

Step 3 - Update Route Tables

In this step, I will update the route tables in both VPCs to point traffic destined for the other VPC through the peering connection, because without these routes the two VPCs have no idea how to reach each other even though the peering connection exists

Step 4 - Launch EC2 Instances

In this step, I will launch an EC2 instance in each VPC, because I need something running inside each network to test whether the peering connection is actually working



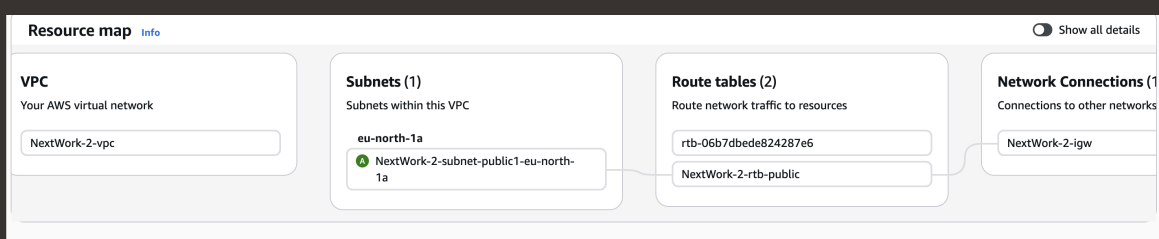
Multi-VPC Architecture

I started my project by launching two VPCs using the visual VPC resource map. I created two subnets in total, one public subnet inside each VPC

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16. They have to be unique because if the IP ranges overlapped, the VPC peering connection would fail as AWS wouldn't be able to route traffic between them correctly.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as I won't be SSHing into them directly. I'll be using EC2 Instance Connect instead to test the peering connection





VPC Peering

A VPC peering connection is a networking link between two VPCs that allows them to communicate with each other using private IP addresses, as if they were part of the same network

VPCs would use peering connections to securely share resources between isolated networks without routing traffic over the public internet, for example connecting a production environment to a logging or monitoring VPC

The difference between a Requester and an Acceptor in a peering connection is that the Requester is the VPC that initiates the connection, and the Acceptor is the VPC that receives and approves it. Both sides have to agree for the connection to be established

Select another VPC to peer with

Account

☒ My account

☐ Another account

Region

☒ This Region (eu-north-1)

☐ Another Region

VPC ID (Acceptor)

vpc-0c8f254bc8f982e9d (NextWork-2-vpc)

VPC CIDRs for vpc-0c8f254bc8f982e9d (NextWork-2-vpc)

CIDR	Status	Status reason
10.2.0.0/16	Associated	-

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Q Name

Value - optional

Q VPC 1 <-> VPC 2

Remove



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because the peering connection alone doesn't automatically direct traffic. The route tables need explicit rules telling each VPC to send traffic destined for the other VPC's CIDR block through the peering connection

My VPCs' new routes have a destination of the other VPC's CIDR block, so 10.2.0.0/16 for VPC 1 and 10.1.0.0/16 for VPC 2. The routes' target was the VPC peering connection ID.

Routes (3)

Filter routes

Destination ▾	Target ▾	Status ▾	Propag... ▾	Route Origin
0.0.0.0/0	igw-00208...	✓ Active	No	Create Route
10.1.0.0/16	pcx-0fc0d4...	✓ Active	No	Create Route
10.2.0.0/16	local	✓ Active	No	Create Route Table



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will connect to my first EC2 instance using EC2 Instance Connect and fix a connection error, because I need to get inside the instance to run connectivity tests against the second VPC

Step 6 - Connect to EC2 Instance 1

In this step, I will connect to Instance 1 again using EC2 Instance Connect and fix another error, because even with an Elastic IP assigned there is still something blocking the connection that needs to be resolved

Step 7 - Test VPC Peering

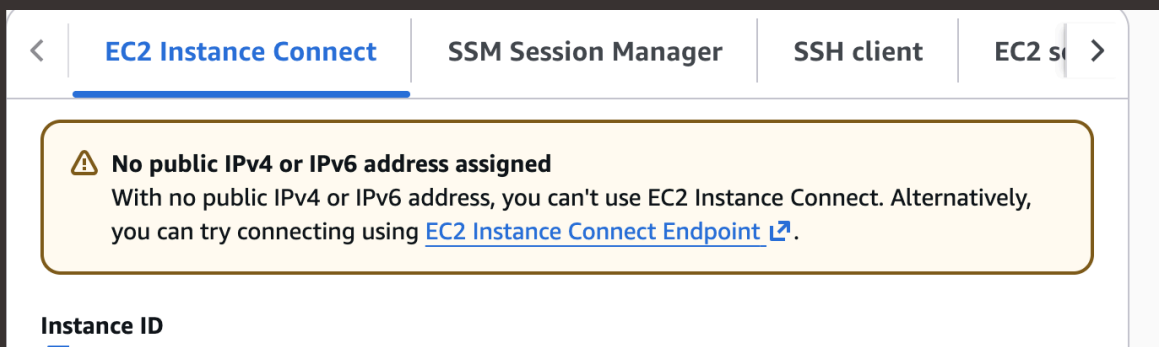
In this step, I will use Instance 1 to send ping messages to Instance 2 and troubleshoot any errors until Instance 2 successfully responds, because this is how I verify that the VPC peering connection is actually routing traffic correctly between the two networks

JJ

Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to connect to my EC2 instance directly from the browser, because it's a quick and simple way to access an instance without needing to set up key pairs or an SSH client

I was stopped from using EC2 Instance Connect as my EC2 instance didn't have a public IP address assigned, which meant EC2 Instance Connect had no way to reach it from outside the VPC





Elastic IP addresses

Elastic IP addresses are static public IP addresses provided by AWS that you can allocate to your account and attach to an EC2 instance, giving it a fixed public IP that persists even if the instance is stopped and restarted

Associating an Elastic IP address resolved the error because it gave my EC2 instance a public IP address, which EC2 Instance Connect needs to establish a connection to the instance

Allocate Elastic IP address [Info](#)

Elastic IP address settings [Info](#)

Public IPv4 address pool

- ☒ Amazon's pool of IPv4 addresses
- ☐ Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) [Learn more](#)
- ☐ Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer owned pools found) [Learn more](#)
- ☐ Allocate using an IPv4 IPAM pool (option disabled because no public IPv4 IPAM pools with AWS service as EC2 were found)

Network border group [Info](#)

Global static IP addresses
AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

Create accelerator

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add new tag



Troubleshooting ping issues

To test VPC peering, I ran the command ping followed by the private IP address of Instance 2, which sends ICMP packets from Instance 1 across the peering connection to check if Instance 2 responds

A successful ping test would validate my VPC peering connection because it confirms that traffic is leaving Instance 1, travelling across the peering connection using the routes I configured, and reaching Instance 2 in the other VPC, which means the entire setup is working end to end

I had to update my second EC2 instance's security group because it was blocking incoming ICMP traffic by default. I added a new rule that allowed all ICMP traffic from the CIDR block of VPC 1, so that the ping requests from Instance 1 could reach Instance 2

```
64 bytes from 10.2.13.15: icmp_seq=795 ttl=127 time=0.232 ms
64 bytes from 10.2.13.15: icmp_seq=796 ttl=127 time=0.264 ms
64 bytes from 10.2.13.15: icmp_seq=797 ttl=127 time=0.196 ms
64 bytes from 10.2.13.15: icmp_seq=798 ttl=127 time=0.202 ms
64 bytes from 10.2.13.15: icmp_seq=799 ttl=127 time=0.191 ms
64 bytes from 10.2.13.15: icmp_seq=800 ttl=127 time=0.200 ms
64 bytes from 10.2.13.15: icmp_seq=801 ttl=127 time=0.177 ms
64 bytes from 10.2.13.15: icmp_seq=802 ttl=127 time=0.196 ms
64 bytes from 10.2.13.15: icmp_seq=803 ttl=127 time=0.184 ms
64 bytes from 10.2.13.15: icmp_seq=804 ttl=127 time=0.181 ms
64 bytes from 10.2.13.15: icmp_seq=805 ttl=127 time=0.180 ms
64 bytes from 10.2.13.15: icmp_seq=806 ttl=127 time=0.196 ms
64 bytes from 10.2.13.15: icmp_seq=807 ttl=127 time=0.205 ms
64 bytes from 10.2.13.15: icmp_seq=808 ttl=127 time=0.190 ms
64 bytes from 10.2.13.15: icmp_seq=809 ttl=127 time=0.252 ms
64 bytes from 10.2.13.15: icmp_seq=810 ttl=127 time=0.191 ms
64 bytes from 10.2.13.15: icmp_seq=811 ttl=127 time=0.239 ms
64 bytes from 10.2.13.15: icmp_seq=812 ttl=127 time=0.242 ms
64 bytes from 10.2.13.15: icmp_seq=813 ttl=127 time=0.180 ms
64 bytes from 10.2.13.15: icmp_seq=814 ttl=127 time=0.186 ms
64 bytes from 10.2.13.15: icmp_seq=815 ttl=127 time=0.198 ms
64 bytes from 10.2.13.15: icmp_seq=816 ttl=127 time=0.195 ms
64 bytes from 10.2.13.15: icmp_seq=817 ttl=127 time=0.196 ms
64 bytes from 10.2.13.15: icmp_seq=818 ttl=127 time=0.234 ms
64 bytes from 10.2.13.15: icmp_seq=819 ttl=127 time=0.182 ms
64 bytes from 10.2.13.15: icmp_seq=820 ttl=127 time=0.184 ms
64 bytes from 10.2.13.15: icmp_seq=821 ttl=127 time=0.213 ms
64 bytes from 10.2.13.15: icmp_seq=822 ttl=127 time=0.204 ms
64 bytes from 10.2.13.15: icmp_seq=823 ttl=127 time=0.202 ms
64 bytes from 10.2.13.15: icmp_seq=824 ttl=127 time=0.261 ms
64 bytes from 10.2.13.15: icmp_seq=825 ttl=127 time=0.181 ms
64 bytes from 10.2.13.15: icmp_seq=826 ttl=127 time=0.182 ms
64 bytes from 10.2.13.15: icmp_seq=827 ttl=127 time=0.186 ms
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

